

✓ PRACTICAL NO 2

Name : Pranjal somatkar roll no. B- 48

AIM: Data Ingestion and Cleaning: Acquire data and prepare it for analysis

✓ Data Ingestion and Cleaning:

1. What is data ingestion and why is it important? Which Python library is used?

Answer:

- **What is Data Ingestion?** Data ingestion is the process of collecting, importing, and loading raw data from various sources into a system (like a database, data warehouse, or a DataFrame in memory) where it can be stored, processed, and analyzed. These sources can be diverse, including:
 - **Structured Data:** CSV files, SQL databases, Excel spreadsheets.
 - **Semi-structured Data:** JSON, XML files, logs.
 - **Unstructured Data:** Images, videos, text documents (though often processed for ingestion in a structured way). In data science, it's the fundamental first step in any data pipeline.
- **Why is Data Ingestion Important?** Effective data ingestion is crucial because it:
 - **Ensures Data Availability:** Makes data accessible in a usable and structured format for downstream tasks.
 - **Integrates Diverse Sources:** Combines data from multiple, disparate sources into a unified system.
 - **Maintains Consistency:** Helps maintain data consistency and completeness across the system.
 - **Enables Processing:** Facilitates real-time or batch processing, depending on the requirements.
 - **Forms Analysis Basis:** Provides the necessary foundation for accurate data analysis, reporting, and machine learning model development. Poor ingestion can lead to flawed insights.
- **Which Python Library is Used?** In Python, the **Pandas** library is predominantly used for data ingestion and initial manipulation due to its powerful `DataFrame` structure and versatile I/O tools.

Examples of Pandas Ingestion Functions:

```
import pandas as pd

# Loading a CSV file
df_csv = pd.read_csv("data.csv")

# Loading an Excel file
df_excel = pd.read_excel("data.xlsx")

# Loading a JSON file
df_json = pd.read_json("data.json")
```

2. Why is data cleaning necessary?

Answer:

- **What is Data Cleaning?** Data cleaning (or data scrubbing) is the process of detecting and correcting (or removing) corrupt, inaccurate, incomplete, or irrelevant records from a dataset. It involves identifying and handling various data quality issues to ensure the data is fit for analysis.
- **Why is it Necessary? (Common Data Issues & Consequences)** Real-world datasets are rarely perfect and often contain issues that can severely impact analysis and model performance. Data cleaning is necessary because:
 - **Missing Values:** Many datasets have missing entries (e.g., an empty cell in an 'Age' column). If not addressed, statistical calculations or machine learning algorithms might fail or produce biased results.
 - **Duplicate Records:** Identical entries can skew counts, averages, and lead to overrepresentation of certain data points.
 - **Inconsistent Formats:** Data might be stored in different units or formats (e.g., 'USA', 'United States', 'U.S.' for the same country), making aggregation and comparison difficult.
 - **Incorrect/Erroneous Data:** Typos, incorrect data entry, or values outside a valid range (e.g., age = 200).
 - **Outliers/Noise:** Extreme values that might represent measurement errors or rare events, which can distort statistical models.

Consequences of Unclean Data:

- **Inaccurate Analysis:** Leading to incorrect conclusions and poor business decisions.
- **Flawed Machine Learning Models:** Models trained on dirty data will learn incorrect patterns, resulting in low accuracy and unreliable predictions.

- **Operational Inefficiencies:** Data issues can complicate data integration and processing workflows.
- **Benefits of Data Cleaning:**
 - **Improved Data Quality:** Ensures accuracy, completeness, consistency, and validity of the data.
 - **Increased Accuracy of Analysis:** Leads to more reliable insights and better decision-making.
 - **Enhanced ML Model Performance:** Models perform better when trained on clean, high-quality data.
 - **Reduced Bias:** Minimizes bias introduced by erroneous or inconsistent data.

Example (Handling Missing Values):

```
# Assume 'df' is a DataFrame with a column 'Age' containing missing values
# Fill missing 'Age' values with the median age
df['Age'].fillna(df['Age'].median(), inplace=True)

# Drop rows with any missing values
# df.dropna(inplace=True)

# Drop columns with too many missing values (e.g., more than 70% missing)
# threshold = 0.7 * len(df)
# df.dropna(thresh=threshold, axis=1, inplace=True)
```

3. How do you load a CSV file into a Pandas DataFrame?

Answer:

- **Basic Method:** To load a CSV (Comma-Separated Values) file into a Pandas DataFrame, you use the `pd.read_csv()` function. This function is highly versatile and is the go-to method for reading tabular data from CSV files.

```
import pandas as pd

# Load a CSV file named 'my_data.csv' into a DataFrame called 'df'
df = pd.read_csv("my_data.csv")

# Display the first few rows to confirm loading
print("First 5 rows of the DataFrame:")
print(df.head())
```

- **How it Works:** The `read_csv()` function reads the CSV file line by line, interprets the commas (or other specified delimiters) as separators between columns, and organizes the data into a structured Pandas DataFrame. Each row in the CSV becomes a row in the DataFrame, and each column becomes a series.
- **Common Optional Parameters:** `pd.read_csv()` offers numerous parameters to handle different CSV formats and specific loading requirements:
 - `sep` or `delimiter`: Specifies the column separator (default is ',').
 - `header`: Row number to use as the column names (default is 0, the first row).
 - `names`: List of column names to use.
 - `index_col`: Column(s) to use as the row labels of the DataFrame.
 - `skiprows`: Line numbers to skip (0-indexed) or number of lines to skip at the start of the file.
 - `na_values`: Additional strings to recognize as `NaN` (missing values).
 - `dtype`: Dictionary specifying data types for columns.
 - `parse_dates`: List of column names to parse as dates.

Example (with optional parameters):

```
# Assuming a CSV where the first column is an ID and should be the index
# and 'N/A' should also be treated as a missing value
df_custom = pd.read_csv("another_data.csv", index_col=0, na_values=['N/A', 'U'])
print("\nDataFrame loaded with custom index and na_values:")
print(df_custom.head())
```

4. What is the difference between `head()` and `tail()`?

Answer:

Both `head()` and `tail()` are essential Pandas DataFrame methods used for quickly inspecting the contents of a DataFrame, but they serve opposite purposes:

- `df.head(n=5)`:
 - **Purpose:** Displays the **first `n` rows** of the DataFrame. By default, if no `n` is specified, it shows the first 5 rows.
 - **Use Cases:**
 - To get a quick overview of the DataFrame's structure.
 - To check column names and data types.
 - To ensure data was loaded correctly.

- To see the beginning of the dataset.

Example:

```
# Display the first 5 rows (default)
print("\nFirst 5 rows (df.head()):")
print(df.head())

# Display the first 10 rows
print("\nFirst 10 rows (df.head(10)):")
print(df.head(10))
```

- **df.tail(n=5):**
 - **Purpose:** Displays the **last n rows** of the DataFrame. By default, if no **n** is specified, it shows the last 5 rows.
 - **Use Cases:**
 - To check for recent additions to the dataset.
 - To identify any potential anomalies or issues at the end of the data (e.g., footers, summaries).
 - To verify that data processing steps have correctly appended or modified the end of the DataFrame.

Example:

```
# Display the last 5 rows (default)
print("\nLast 5 rows (df.tail()):")
print(df.tail())

# Display the last 3 rows
print("\nLast 3 rows (df.tail(3)):")
print(df.tail(3))
```

5. How can you identify missing values in a dataset?**Answer:**

Missing values are common in real-world datasets and must be identified and handled appropriately. In Pandas, missing values are typically represented by **NaN** (Not a Number) for numerical data or **None** for object-type data.

Here are the primary methods to identify missing values:

1. **df.isnull()** or **df.isna()**:

- **Description:** Both functions return a Boolean DataFrame of the same shape as the original DataFrame, where `True` indicates a missing value (`NaN` or `None`) and `False` indicates a non-missing value.
- **Use Case:** Useful for visually inspecting the presence of missing values across the entire DataFrame or for logical indexing.
- **Example:**

```
# Create a sample DataFrame with missing values
data = {'A': [1, 2, None, 4], 'B': [5, 6, 7, pd.NA], 'C': [8, 9, 10, 11]}
sample_df = pd.DataFrame(data)
sample_df.loc[0, 'C'] = pd.NA

print("\nSample DataFrame with missing values:")
print(sample_df)

print("\nBoolean DataFrame indicating missing values:")
print(sample_df.isnull())
```

2. `df.isnull().sum()`:

- **Description:** This method is often chained with `.sum()` after `isnull()` to provide a count of missing values for each column. If `axis=1` is specified, it counts missing values per row.
- **Use Case:** Provides a concise summary of how many missing values exist in each column, which is essential for prioritizing cleaning efforts.
- **Example:**

```
print("\nCount of missing values per column:")
print(sample_df.isnull().sum())
```

3. `df.isnull().any()`:

- **Description:** When chained with `.any()`, it returns a Boolean Series, where `True` indicates that the respective column (or row, if `axis=1`) contains at least one missing value.
- **Use Case:** Quickly identifies which columns have *any* missing data, without needing the exact count.
- **Example:**

```
print("\nColumns with any missing values:")
print(sample_df.isnull().any())
```

4. `df.info()`:

- **Description:** The `.info()` method provides a concise summary of a DataFrame, including the data types of each column, the number of non-null values, and memory usage. The 'Non-Null Count' is particularly useful for identifying columns with missing data.
- **Use Case:** A quick way to get an overview of the DataFrame's structure, data types, and simultaneously spot columns with fewer non-null entries than the total number of rows.
- **Example:**

```
print("\nDataFrame info (showing non-null counts):")
sample_df.info()
```

Example Insight (from `df.info()`): If a DataFrame has 100 rows, and `df.info()` shows a 'Non-Null Count' of 80 for 'Column_A', it immediately tells you that 'Column_A' has 20 missing values.

LOAD DATASET

```
import pandas as pd

# Load dataset
df = pd.read_csv('/content/Iris[1].csv')

# Display first 5 rows
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	
0	1	5.1	3.5	1.4	0.2	Iris-setosa	
1	2	4.9	3.0	1.4	0.2	Iris-setosa	
2	3	4.7	3.2	1.3	0.2	Iris-setosa	
3	4	4.6	3.4	1.5	0.2	Iris-setosa	
4	5	5.0	3.5	1.6	0.2	Iris-setosa	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Understand Data (MANDATORY)

```
df.info()
df.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     150 non-null   int64
1   SepalLengthCm         150 non-null   float64
2   SepalWidthCm          150 non-null   float64
3   PetalLengthCm         150 non-null   float64
4   PetalWidthCm          150 non-null   float64
5   Species               150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
      'Species'],
      dtype='object')
```

1. Select a single column

```
df['SepalLengthCm']
```

	SepalLengthCm
0	5.1
1	4.9
2	4.7
3	4.6
4	5.0
...	...
145	6.7
146	6.3
147	6.5
148	6.2
149	5.9

150 rows × 1 columns

dtype: float64

2. Add a new column with constant values


```
df['Source'] = 'Iris Dataset'
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Source
0	1	5.1	3.5	1.4	0.2	Iris-setosa	Iris Dataset
1	2	4.9	3.0	1.4	0.2	Iris-setosa	Iris Dataset
2	3	4.7	3.2	1.3	0.2	Iris-setosa	Iris Dataset

Next steps: [Generate code with df](#) [New interactive sheet](#)

3. Sort DataFrame by one column

```
df_sorted = df.sort_values(by='SepalLengthCm', ascending=False)
df_sorted.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Source
131	132	7.9	3.8	6.4	2.0	Iris-virginica	Data
122	123	7.7	2.8	6.7	2.0	Iris-virginica	Data
118	119	7.7	2.6	6.9	2.3	Iris-virginica	Data

Next steps: [Generate code with df_sorted](#) [New interactive sheet](#)

4. Rename DataFrame columns

```
df.rename(columns={
    'SepalLengthCm': 'Sepal_Length',
    'SepalWidthCm': 'Sepal_Width'
})
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Source
0	1	5.1	3.5	1.4	0.2	Iris-setosa	Iris Dataset
1	2	4.9	3.0	1.4	0.2	Iris-setosa	Iris Dataset
2	3	4.7	3.2	1.3	0.2	Iris-setosa	Iris Dataset

Next steps: [Generate code with df](#) [New interactive sheet](#)

5. Summary statistics

df.describe()

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

6. Filter rows using multiple conditions

```
filtered_df = df[(df['SepalLengthCm'] > 5.0) & (df['PetalLengthCm'] > 4.0)]
filtered_df
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	So
50	51	7.0	3.2	4.7	1.4	Iris-versicolor	Da
51	52	6.4	3.2	4.5	1.5	Iris-versicolor	Da
52	53	6.9	3.1	4.9	1.5	Iris-versicolor	Da
54	55	6.5	2.8	4.6	1.5	Iris-versicolor	Da
55	56	5.7	2.8	4.5	1.3	Iris-versicolor	Da
...	...	...	...	...	...	...	
145	146	6.7	3.0	5.2	2.3	Iris-virginica	Da
146	147	6.3	2.5	5.0	1.9	Iris-virginica	Da

Next steps:

[Generate code with filtered_df](#)

[New interactive sheet](#)

7. Group by categorical column & calculate mean

```
grouped = df.groupby('Species')['SepalLengthCm'].mean()
grouped
```

SepalLengthCm	
Species	
Iris-setosa	5.006
Iris-versicolor	5.936
Iris-virginica	6.588

dtype: float64

8. Merge two DataFrames

```
df2 = pd.DataFrame({
    'Id': df['Id'],
    'Region': ['India'] * len(df)
})
```

https://colab.

1 / 14

1

```
# Merge on common column 'Id'
merged_df = pd.merge(df, df2, on='Id')
merged_df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Source
0	1	5.1	3.5	1.4	0.2	Iris-setosa	Iris Dataset
1	2	4.9	3.0	1.4	0.2	Iris-setosa	Iris Dataset
2	3	4.7	3.2	1.3	0.2	Iris-setosa	Iris Dataset

Next steps:

[Generate code with merged_df](#)

[New interactive sheet](#)

9. Drop rows and columns

```
# Drop column
df = df.drop(columns=['Source'])
# Drop rows (example: first row)
df = df.drop(index=0)
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	
1	2	4.9	3.0	1.4	0.2	Iris-setosa	
2	3	4.7	3.2	1.3	0.2	Iris-setosa	
3	4	4.6	3.1	1.5	0.2	Iris-setosa	
4	5	5.0	3.6	1.4	0.2	Iris-	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

10. Apply a simple function

```
def size_category(PetalLengthCm):
    if PetalLengthCm > 5:
        return 'Large'
    elif PetalLengthCm > 3:
        return 'Medium'
    else:
        return 'Small'
```

```
df['Petal_Size'] = df['PetalLengthCm'].apply(size_category)
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species	Petal_
1	2	4.9	3.0	1.4	0.2	Iris-setosa	\$
2	3	4.7	3.2	1.3	0.2	Iris-setosa	\$
3	4	4.6	3.1	1.5	0.2	Iris-setosa	\$

Next steps:

[Generate code with df](#)[New interactive sheet](#)

✓ Conclusion:

The discussions around **data ingestion, cleaning, and initial inspection** underscore their critical role as the foundational pillars of any successful data analysis or machine learning project. These steps are not merely preparatory; they directly dictate the quality, reliability, and ultimately, the utility of any insights derived from the data.

1. Data Ingestion: The Gateway to Insights

- We established that data ingestion is the crucial first step, involving the collection and loading of raw data from diverse sources into a usable format, typically a Pandas DataFrame in Python. Its importance cannot be overstated, as a flawed ingestion process can corrupt the entire data pipeline. Libraries like Pandas, with functions such as `pd.read_csv()`, `pd.read_excel()`, and `pd.read_json()`, provide robust tools for handling various data formats, making this initial phase efficient and manageable.

2. Data Cleaning: Ensuring Data Integrity

- The necessity of data cleaning stems from the inherent messiness of real-world datasets. Issues like missing values, duplicates, inconsistent formats, and outliers are pervasive. Neglecting these can lead to biased models, inaccurate analyses, and ultimately, poor decision-making. Data cleaning is about meticulously detecting and addressing these anomalies, transforming raw, imperfect data into a clean, consistent, and reliable asset. Techniques like filling or dropping missing values are fundamental to this process.

3. Initial Data Inspection: First Glimpse of the Truth

- Methods such as `df.head()` and `df.tail()` are invaluable for rapidly inspecting the DataFrame's structure, column names, and the first and last few records. These quick checks help verify successful data loading and can offer early clues about potential data quality issues, such as unexpected footers or headers, or data truncation. They serve as an immediate sanity check for the ingested data.

4. Identifying Missing Values: A Prerequisite for Cleaning

- Before cleaning can effectively begin, missing values must be systematically identified. Pandas provides powerful tools like `df.isnull()`, `df.isnull().sum()`, `df.isnull().any()`, and `df.info()` for this purpose. These functions allow analysts to quickly quantify missingness per column, pinpoint columns with any missing data, and gain an overview of non-null counts, guiding subsequent cleaning strategies.

In essence, **data ingestion provides the raw material, cleaning purifies it, and inspection offers the initial quality control.** Together, these processes ensure that the data is not only accessible but also reliable and fit for purpose, enabling accurate analysis, robust model development, and trustworthy data-driven outcomes.

Start coding or [generate](#) with AI.